

SOFTWARE MODELLING

DIAN LE ROUX – 25147065 / MARKO DE SWARDT – 24658562 / JAY MACASKILL – 25198387

1 SEPTEMBER 2026 / COS 214 PRACTICAL 3 (GAMEFEST)

Prelude

7.3) Push or pull model

We have chosen to use the push model whereby the Subject sends relevant information to update(...), we pass the notice into the update method which allows the Observer to immediately react. There is no back and forth, the Subject tells the Observer there is an evacuate notice (for example), then the Observer can immediately respond by closing, without needing to re-query any information on the state from the Subject again. This is more efficient, and in the context of the overall system, re-querying would not be logical. For instance, if there were a real-world event, the Control Desk would not say “something is wrong” then wait for the Observers to say “what?” instead, the Control Desk would say “there is [emergency], evacuate immediately” and the push model of the Observer pattern models this more accurately than the pull model.

Task 1

1.1) Define your event (GameFest) in approximately one page or less.

Here we have a gaming convention, GameFest as our chosen event. Inside GameFest, we have eight concrete event units, namely the Ticket Gate, the eSports Arena, the Queue, the Main Stage, the Arcade Row, Pinball Alley, the Merch Stall, the Demo Station and the Cosplay Corner, which will later serve as Leaves (the Leaf participant is EventUnit). We also have six event groups, whereby three are larger event groups and one is nested inside of another, with another nested therein and one nests all of the others within itself, providing four layers of nesting, namely the Main Hall, the Tournament Wing, the Vendor Hall, the Retro Corner (nested within the Classic Games hall) and GameFest itself. This is a typical gaming convention, we have a Main Stage for any talks, performances, and so on as well as a Ticket Gate and a Cosplay Corner for cosplay competitions, all of which are contained in the Main Hall. In the Main Hall, we also have a Classic Games hall, containing a Retro Corner with a row of arcade machines and a pinball alley. Additionally, this convention is open to vendors, who are based in the Vendor Hall with their Demo Stations and Merch Stalls. We also have a Tournament Wing with a Queue for eSports tournaments taking place in the eSports Arena. All of this, together with the Control Desk sending out notices to keep everyone safe, makes GameFest

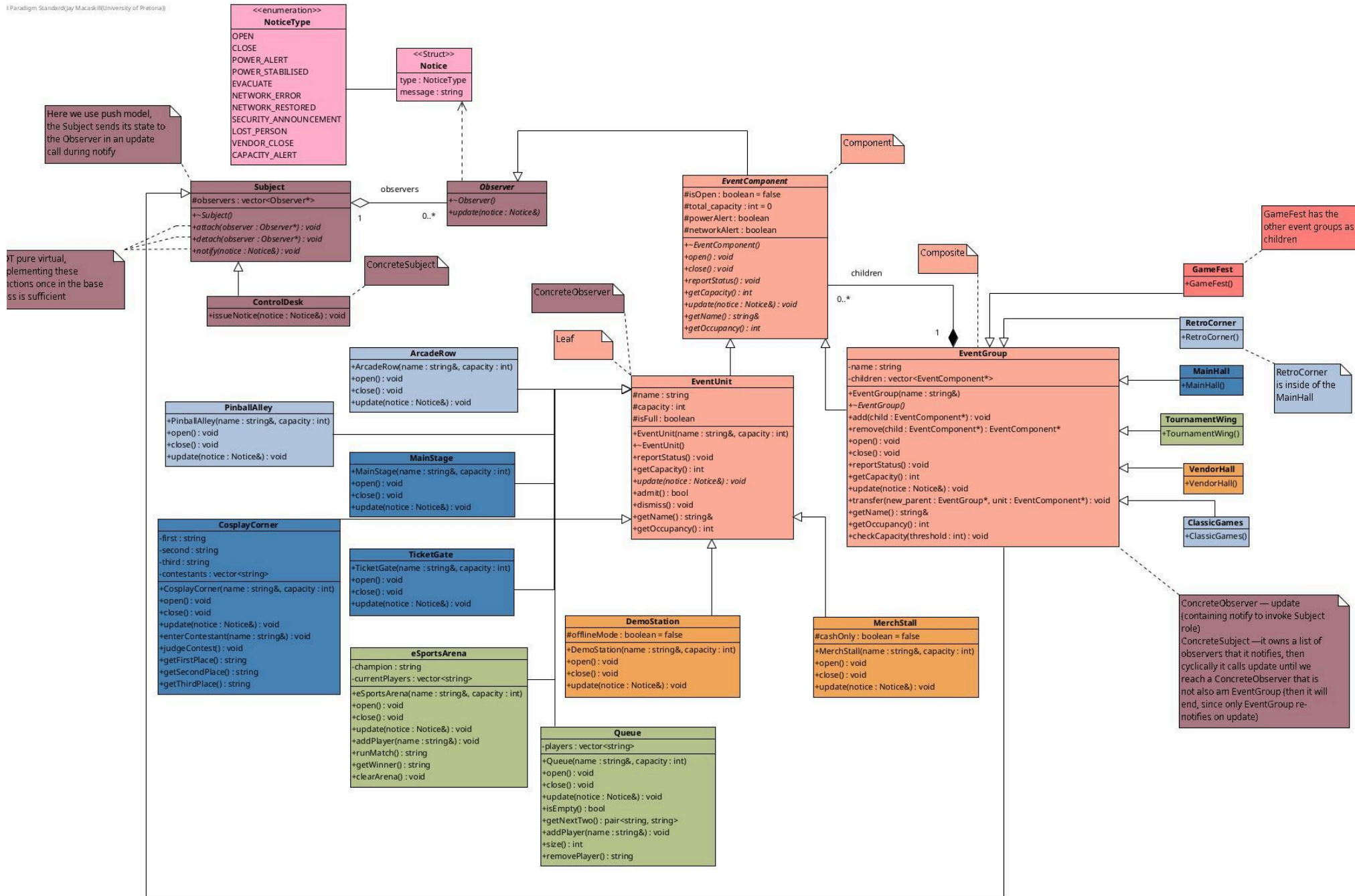
1.2) Identify the GoF participants in both Observer and Composite in your design.

Observer	Composite
----------	-----------

Observer	We have defined the abstract Observer, Observer	Component	We have our EventComponent
Subject	We have defined the abstract Subject, Subject	Leaf	We have our EventUnit (ArcadeRow, MainStage, CosplayCorner, PinballAlley, TicketGate, eSportsArena, Queue, DemoStation and MerchStall are EventUnits)
ConcreteObserver	We have multiple ConcreteObservers (by means of inheritance), each EventComponent (EventUnit, EventGroup) is a ConcreteObserver, since EventComponent inherits from Observer (since they need to be sent notices)	Composite	EventGroup (we have MainHall, RetroCorner, TournamentWing, VendorHall, ClassicGames, and GameFest being EventGroups which contain other EventGroups or EventUnits, or even an EventGroup which contains EventGroup(s))
ConcreteSubject	We have ControlDesk being our ConcreteSubject, as well as EventGroup, since EventGroup will have its own Observers due to nesting, it will need to notify down a list to its children, so it is also a ConcreteSubject		

1.3) Complete UML class diagram

I Paradigm Standard(jay Macaskill@University of Pretoria)



1.4) Design rationale

a) Why Composite is a genuine part-whole tree

Every EventGroup contains zero or more EventComponent objects, these can either be individual EventUnit (leaves) or other EventGroup (composites). This creates a tree structure where each node can contain other nodes, and operations performed on a parent automatically go down to all its descendants, creating a part-whole tree. Here, ownership is clear, an EventComponent belongs to one parent at a time, operations are recursive. Calling close() on MainHall closes MainStage, TicketGate, ClassicGames and CosplayCorner as well. We also have uniform treatment, all parts are treated the same as a whole. EventGroup and EventUnit are treated identically through the EventComponent interface.

b) Why Observer is required rather than hard-coded calls

The notification system is a one-to-many dependency, we have many classes which need to know about one notification, it would be bad to do hard-coded calls. If we had hard-coded calls instead of sending a notice, we would have to create a function for each type of notice (e.g. evacuate) and this function would need to know about all the components that need to react to the notice, creating tight coupling, and it would also need to know exactly how each unit should react. This violates the open/closed principle, we can't extend the system without modifying existing code and it reduces flexibility. Also when it comes to the children, there would need to be manual cascading of notifications to the children instead of doing so recursively.

c) Who owns event components

EventComponents are owned by their parent EventGroup through the children vector. This is strict, exclusive ownership. Each EventComponent has exactly one parent that manages its lifetime, this mirrors real-world behaviour such as one Stage not being in two Halls, or a Hall being destroyed and the Stage not being able to persist. The EventGroup is responsible for destroying its children, which ensures the tree is cleaned up recursively. If an EventComponent moves between groups, we must handle this carefully with the transfer function. GameFest, however, is owned by main and is destroyed at the program end (through an explicit delete game_fest; call) which cascades through the tree.

d) Who owns or does not own Observer references

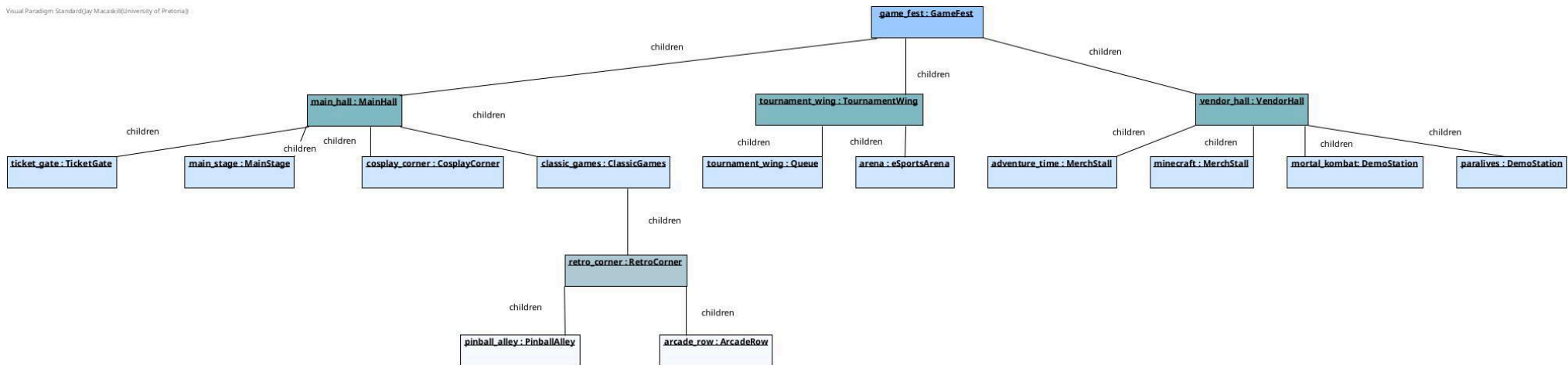
Observer references are non-owning because they have separate lifetimes. An Observer can observe multiple Subjects, if Subjects owned their Observers then an object with multiple Subjects would have multiple owners which would cause double deletion. For instance, MainHall appears in children of GameFest and observers of GameFest, in the children vector, it is owned, in the observers vector, it is not owned. If observers owned the pointers, then MainHall would be deleted twice.

e) Why any class that is both Subject and Observer is not a misuse of either pattern

EventGroup is both a Subject and an Observer, this is not a misuse of the pattern. EventGroup is an Observer receiving notifications from above, reacting to changes. (e.g ControlDesk -> GameFest -> MainHall, so MainHall is an Observer to GameFest and GameFest is an Observer to ControlDesk), but also EventGroup sends notifications below (like how GameFest must send the notification over to MainHall, MainHall must send down to CosplayCorner, MainStage, TicketGate, ClassicGames and RetroCorner (an EventGroup) observes but also must send the notification down to ArcadeRow through the hierarchy. So we see that each EventGroup is actually an Observer and a Subject, each time it is notified, it has to re-notify all its Observers. There is no pattern contamination, since the Composite children vector and the Observer observers vector are two different vectors managed differently. As an Observer, EventGroup receives from parents, as a Subject, EventGroup sends to children – these are different problems being solved and it enables notifications to tumble down to all the children, from root to leaves. Complementary patterns also exist, and we have seen it before in this course where patterns compose/mix with each other to produce an effective system, in this instance Composite defines the structure whilst Observer defines the communication; these two roles do not interfere with each other, they help each other.

Task 2

2.3) Build a sample event with at least three levels of nesting. Draw an object diagram showing the concrete object instances and the ownership tree



```
// --- Leaves (Level 4) ---
```

```
TicketGate* ticket_gate = new TicketGate("Ticket Gate", 35);
```

```
MainStage* main_stage = new MainStage("Main Stage", 25);
```

```
CosplayCorner* cosplay_corner = new CosplayCorner("Cosplay Corner", 20);
```

```
ArcadeRow* arcade_row = new ArcadeRow("Arcade Row", 5);
```

```
PinballAlley* pinball_alley = new PinballAlley("Pinball Alley", 5);
```

```
Queue* tournament_queue = new Queue("Tournament Queue", 30);
```

```
eSportsArena* arena = new eSportsArena("eSportsArena", 10);
```

```
MerchStall* adventure_time = new MerchStall("Adventure Time", 15);
```

```
MerchStall* minecraft = new MerchStall("Minecraft", 15);
```

```
DemoStation* mortal_kombat = new DemoStation("Mortal Kombat", 8);
```

```
DemoStation* paralives = new DemoStation("Paralives", 5);
```

```
// --- Sub-areas (Level 4) ---
RetroCorner* retro_corner = new RetroCorner;
    retro_corner->add(arcade_row);
    retro_corner->add(pinball_alley);

// --- Sub-areas (Level 3) ---
ClassicGames* classic_games = new ClassicGames;
    classic_games->add(retro_corner);

// --- Halls/Wings (Level 2) ---
MainHall* main_hall = new MainHall;
    main_hall->add(classic_games);
    main_hall->add(ticket_gate);
    main_hall->add(main_stage);
    main_hall->add(cosplay_corner);

TournamentWing* tournament_wing = new TournamentWing;
    tournament_wing->add(tournament_queue);
    tournament_wing->add(arena);

VendorHall* vendor_hall = new VendorHall;
    vendor_hall->add(adventure_time);
    vendor_hall->add(minecraft);
    vendor_hall->add(mortal_kombat);
    vendor_hall->add(paralives);

// --- Root (Level 1) ---
```

```
GameFest* game_fest = new GameFest; // root (owns everything now)
    game_fest->add(main_hall);
    game_fest->add(tournament_wing);
    game_fest->add(vendor_hall);
```

2.4) Demonstrate destruction of the root and explain why the complete owned subtree is released exactly once. If an object can be transferred between composites, define the ownership rules for that transfer clearly

```
delete game_fest; // how we delete
```

We have the root, game_fest. When we call delete game_fest, then it first goes into its next level and calls delete on all its children (tournament_wing, vendor_hall, main_hall) then, after that those children call delete on their children (retro_corner, anonymous Queue, eSportsArena, the two MerchStalls and DemoStations, PinballAlley and CosplayCorner) who then call delete on their children in their destructor as well, which deletes the anonymous ArcadeRow. So we delete the object that has no parent (the root, game_fest) which results in all of its children deleting their children and so on, until there are no more children to delete (leaf). But also note that we only delete this way for a composite (owning) tree, not for an aggregation tree.

```
vendor_hall->transfer(main_hall, adventure_time); // how we transfer  
  
void transfer(EventGroup* new_parent, EventComponent* unit); // the  
signature of our transfer function  
  
EventComponent* remove(EventComponent* child); // the remove function  
signature
```

Also, we have a transfer function. What this does is it removes the child from the children's vector, then it adds the child to the new parent. So if impromptu they move a stall out of the vendor hall and into the main hall, we first remove the stall from the vendor hall children's array, then we add that to the main hall children's array. What this does is it places the responsibility of cleanup to the new owner, and removes the responsibility from the old owner. Also, the remove function returns the EventComponent that was removed, that way we can handle orphans and we can also handle anonymous objects, the client can then clean up if the component needn't be reattached.

Task 3

3.1) Implement the Subject abstraction and its registration list. attach and detach must behave sensibly for duplicate registration and attempts to detach an observer that is not registered. State your policy

If we attempt to detach an Observer that is not registered, nothing happens and the program safely exists after printing a notification that the Observer is not registered. This means if there is a careless client, the code is non-blocking and the client may continue with other operations without needing to handle an exception, the code silently moves on without breaking anything. If the client tries to register a duplicate, this is also non-blocking. The Observer is not reattached and there's a statement printed to inform the client the Observer is already attached. This means the client does not have to handle any errors. This reflects a real world scenario where the client is not made to handle errors, but the checks are done in the background and a message is given after an illegal task is performed.

3.2) Explain whether observers detach themselves, subjects clean registrations or your design guarantees lifetime ordering in another defensible way

Here we use caller-managed detachment since Observers do not even know which Subjects they're attached to, Subjects can't delete Observers (they don't own them). The caller is responsible for detaching before destruction. Observers remain alive while registered and are detached before destruction.

3.4) Make at least one Composite-level class capable of receiving a notification from above and then notifying interested observers below. Demonstrate a notification cascading through at least three runtime levels

EventGroup implements the Observer and Subject interfaces. As an Observer it receives notifications from its parent or ControlDesk through update, as a Subject it re-broadcasts notifications to its children using notify. So, each EventGroup receives a notice then immediately broadcasts it to all its registered observers (including its children). The cascade involves receiving as an Observer from a parent, then re-broadcasting as a Subject to its children (calls notify again in update). First, we have a tree with multiple levels (root, halls, sub areas, leaves, together with the base level ControlDesk, this is five levels) then we attach the root to the ControlDesk. This means when the root receives a notice, it receives it, then also notifies all its children as well. If the child is an event group, it will also re-broadcast to its children and if it's a leaf then it won't have any children to re-notify so that doesn't happen.

First, the ControlDesk issues the notice (e.g. prints POWER_ALERT: Please stand by while our technicians resolve the power issues) then GameFest receives this, rebroadcasts to MainHall who re-broadcasts to ClassicGames, who re-broadcasts to RetroCorner, who re-broadcasts to ArcadeRow who finally reacts (e.g. Due to a power alert, the arcade will be closing.). This happens similarly in other branches as well, we see that the EventGroups receive and re-broadcast, whilst the EventUnits receive and react. There is no hard-coded logic, just update { notify }

3.5) Choose push or pull for your Observer implementation. Explain the trade-off and show the exact state/message information transferred in your design

```
struct Notice
{
    NoticeType type; /**< The notice type */
    string message; /**< The accompanying message for the notice */
};
```

Here we have a notice struct which chooses a notice type from an enumerable NoticeType. This struct what is passed into the issueNotice, notify and update, as seen in the signatures below.

```
void update(Notice& notice); // Observer
void issueNotice(Notice& notice); // ControlDesk (ConcreteSubject)
```

```
void notify(Notice& notice); // Subject
// This is how a notice is delivered

Notice open;

open.message = "Open the festival!";

open.type = OPEN;

control_desk->issueNotice(open);
```

So instead of having a two way interaction on every notice (as with the pull method), we have a one way interaction on every notice (the push method). This more accurately models a real event. When a control desk delivers a notification, it delivers an entire notice with all the necessary information, instead of saying “there is an update” with no further information and waiting for every unit to ask about the update, it immediately sends all the information. This increases simplicity and allows for looser coupling. We also compare the push model to the pull model in general below:

Push has looser coupling than pull (Observers coupled to the Notice struct only versus Observers knowing the Subject’s interface to every query state); Push has improved performance since one call passes all data whilst with pull Observer may need multiple calls to get all the state information. Push is less complex since the Subject packages all state information upfront whilst the pull model makes the Observer decide what it needs. The push method is also more extensible, new notice types require changing the Notice struct whilst with the pull method new notice types means we have to add new query methods.

It is true that, still, Notice struct must contain all possible data and adding new notice types requires changing the Notice struct, but the event has a fixed set of notice types and the information needed is well-defined and fits cleanly in the Notice struct.

Task 4

4.1) Define at least five event rules that cause concrete units to react differently to the same notice

- POWER_ALERT
 - MainStage dims the lights due to backup power, TicketGate switches to backup scanners, all Queues continue, eSportsArena closes, MerchStall switches to cash-only mode, ArcadeRow closes and DemoStation closes
- NETWORK_ERROR
 - MainStage ignores, TicketGate announces slow processing (backup network services), Queues ignore, eSportsArena delays matches, MerchStall switches to cash-only mode, DemoStation switches to offline mode whilst ArcadeRow ignores as well
- EVACUATE
 - All close with their different exit instructions

- LOST_PERSON
 - Relay the message differently (MainStage announces to the crowd, TicketGate sweeps entrances and exits, etc.)
- VENDOR_CLOSE
 - Only the VendorHall and vendor-related units close (MerchStall, DemoStation) and the MainStage makes an announcement

4.2) Implement at least one runtime reorganisation

Here we have done a transfer function to move one event component to another event group.

```
vendor_hall->transfer(main_hall, adventure_time); // how we transfer
void transfer(EventGroup* new_parent, EventComponent* unit); // the
signature of our transfer function
```

Before vendor_hall owned adventure_time, but now main_hall owns adventure_time. First we remove adventure_time from vendor_hall, then we add adventure_time to main_hall. This automatically keeps Observer notifications in sync. The MerchStall is registered with its new parent and continues to receive notifications through the cascade.

4.3) Implement at least one condition-based decision that will later appear inside an alt or opt combined fragment

Here we have an admit function to admit people into the EventUnits which we use for attendee tracking, we have a isFull and a isOpen flag for the EventUnit, so we use this condition-based decision to admit people

```
if (isFull || !isOpen) { cout << "Sorry! We couldn't admit you right
now."; return false; } // otherwise admit the attendee
```

4.4) Add at least three original features that are not explicitly required above

1. Tournament logic

We have added tournament logic by using the rand() function. A player can enter the queue then two players at a time can be added to an eSports arena. To simulate a match taking place, we use the rand() function and whoever gets the highest number wins the match and progresses. This reflects real world tournaments that would occur with winners/losers and advancements. Ultimately this leads to an overall champion.

This logic lives entirely in Queue and eSportsArena, nor in EventGroup or ControlDesk, so the shared classes stay unbloated.

2. Best Dressed competition (CosplayCorner)

Attendees can enter a cosplay contest and the CosplayCorner randomly judges first, second, and third place from the entrants (without replacement) once at least three people have entered. This isn't required by either pattern, it's event specific functionality that would exist in a real Best Dressed competition regardless of how the software is structured.

This feature is self contained within CosplayCorner without touching any “god” Observer/Composite machinery

3. Capacity threshold alerting

EventGroup::checkCapacity(threshold) compares the current aggregate occupancy of a whole area against a safety threshold (accurately reflecting real-world safety standards, whereby the capacity of an entire area matters more than the capacity of one unit), and raises a real CAPACITY_ALERT notice that cascades through Observer when it's exceeded (e.g. the Ticket Gate automatically pauses admissions). This reflects genuine safety/crowd-management practice at real venues, independent of any pattern requirement.

This logic only uses the existing Composite traversal and Observer notification mechanisms without adding new responsibilities to those classes

4. Attendance tracking

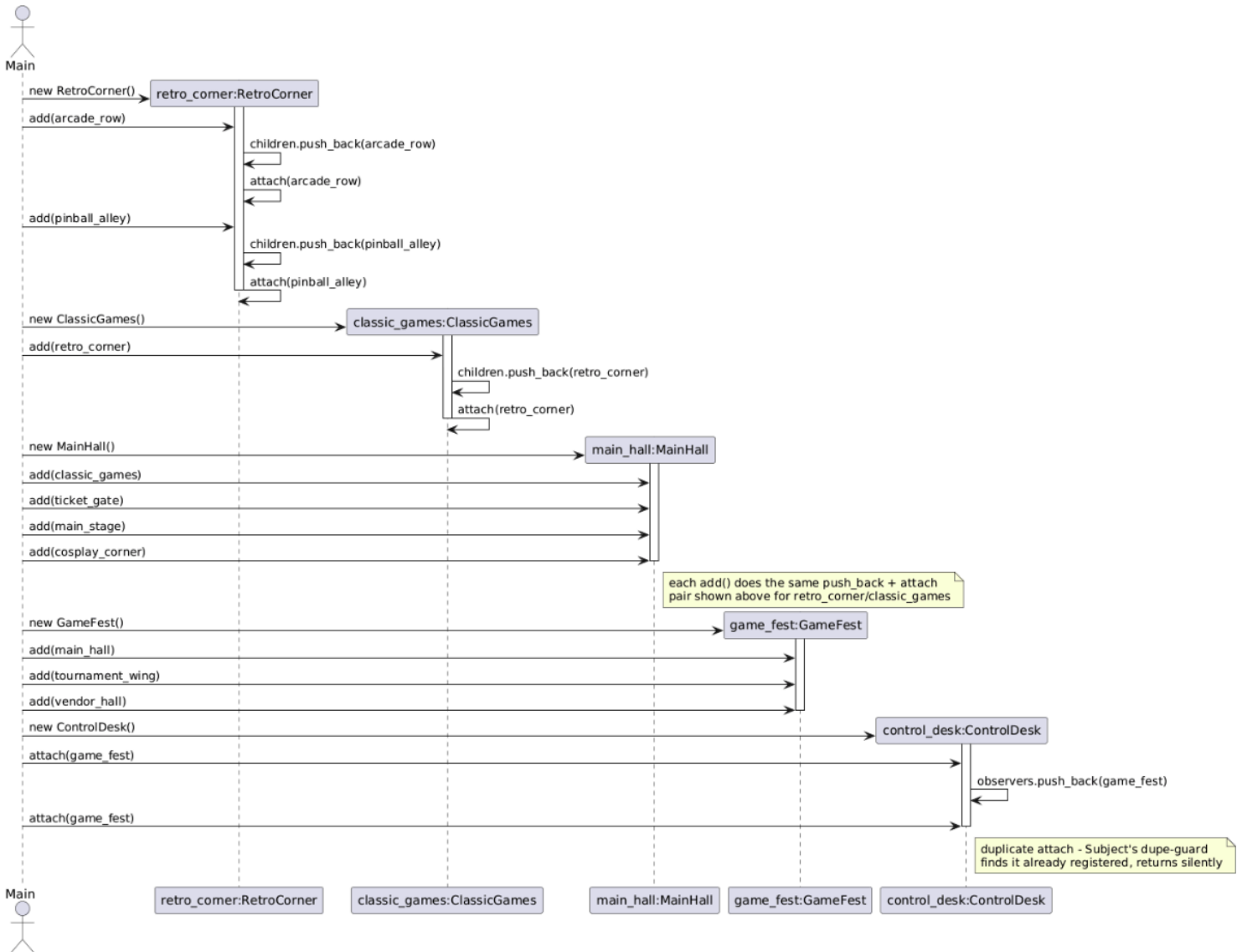
By making use of admit() and dismiss() functions, we are able to live-track the capacity of a specific EventUnit.

This logic is in EventUnit, tracking per-unit capacity without requiring changes to EventGroup, Subject or ControlDesk

Task 5

5.1) SD1 – Building and registering part of the event

SD1 -- Building and Registering Part of the Event



//----- in main.cpp

// --- Sub-areas (Level 4) ---

```
RetroCorner* retro_corner = new RetroCorner;
retro_corner->add(arcade_row);
retro_corner->add(pinball_alley);
```

// --- Sub-areas (Level 3) ---

```
ClassicGames* classic_games = new ClassicGames;
classic_games->add(retro_corner);
```

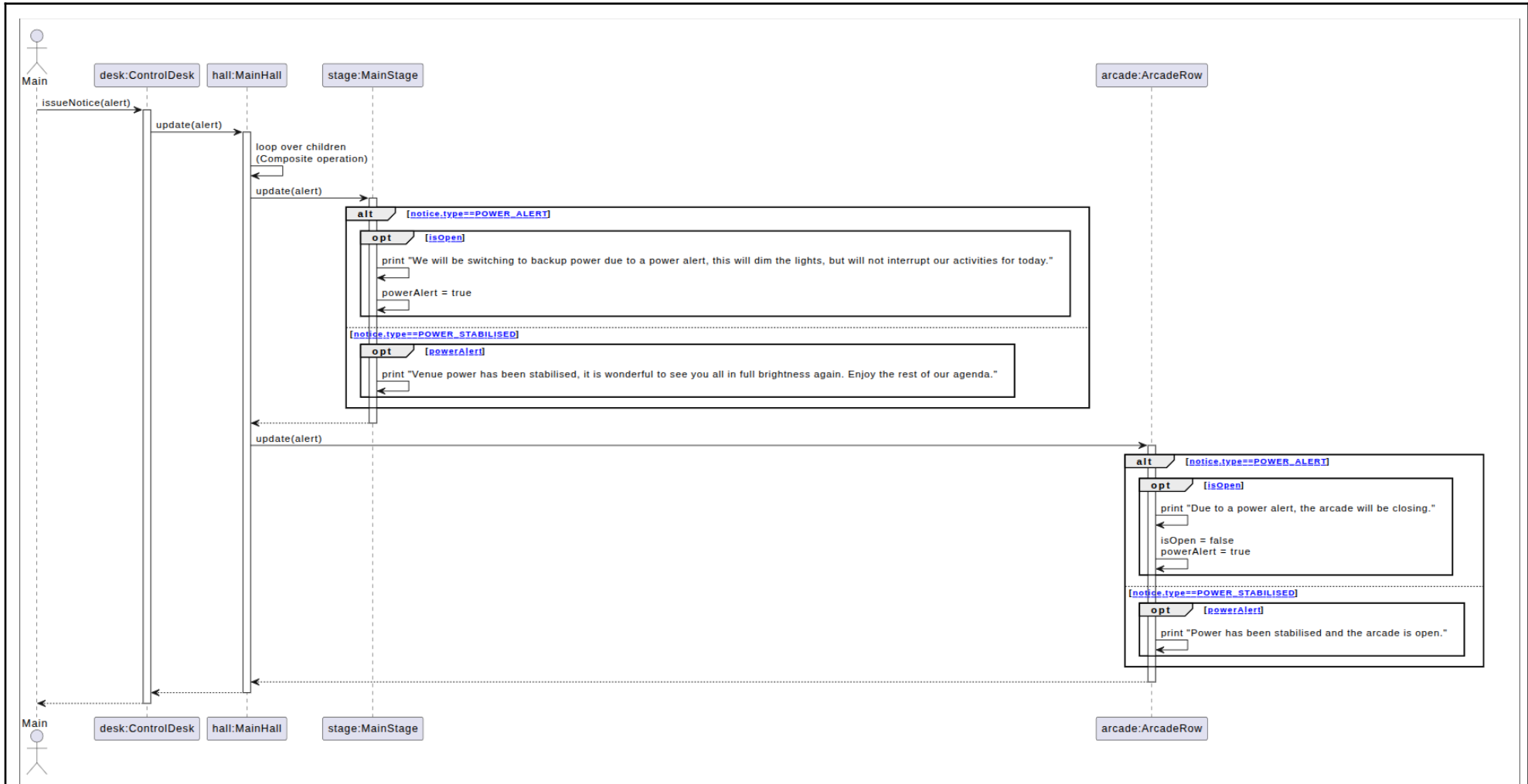
```
// --- Halls/Wings (Level 2) ---
MainHall* main_hall = new MainHall;
    main_hall->add(classic_games);
    main_hall->add(ticket_gate);
    main_hall->add(main_stage);
    main_hall->add(cosplay_corner);

// --- Root (Level 1) ---
GameFest* game_fest = new GameFest;
    game_fest->add(main_hall);
    game_fest->add(tournament_wing);
    game_fest->add(vendor_hall);

// --- Control Desk ---
ControlDesk* control_desk = new ControlDesk;
    control_desk->attach(game_fest);

// ==== HERE WE TEST DUPLICATE ATTACHING! ==== //
    control_desk->attach(game_fest); // this must be handled correctly
```

5.2) SD2 – Cascading event notification



[diagram \(plant uml\)](#)

```

void sd2_powerAlertDemo()
{
    MainStage* stage = new MainStage("Main Stage", 10);

```

```
ArcadeRow* arcade = new ArcadeRow("Arcade Row", 10);
```

```
MainHall* hall = new MainHall();
```

```
hall->add(stage);
```

```
hall->add(arcade);
```

```
ControlDesk* desk = new ControlDesk();
```

```
desk->attach(hall);
```

```
stage->open();
```

```
arcade->open();
```

```
Notice alert;
```

```
alert.type = POWER_ALERT;
```

```
alert.message = "Power alert";
```

```
desk->issueNotice(alert);
```

```
Notice stable;
```

```
stable.type = POWER_STABILISED;
```

```
stable.message = "Power stabilised";
```

```
desk->issueNotice(stable);
```

```
desk->detach(hall);
```

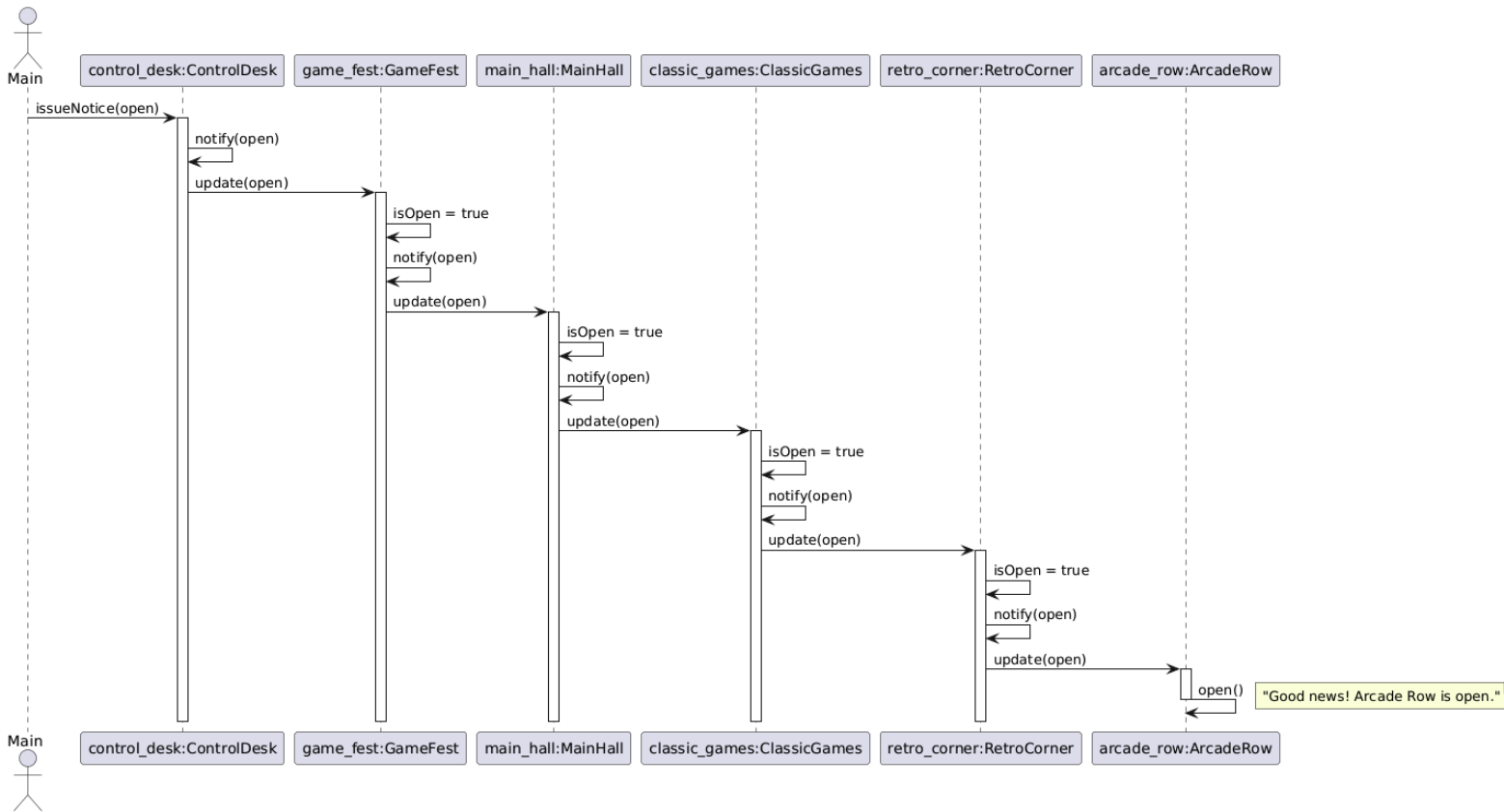
```
delete hall;
```

```
delete desk;
```

```
}
```

5.3) SD3 – Conditional event response and Composite behaviour

SD3 -- Cascading Event Notification (minimal single-branch chain)



```

void sd3_cascadeDemo()
{
    ArcadeRow* arcade = new ArcadeRow("Arcade Row", 5);

    RetroCorner* retro = new RetroCorner();
    retro->add(arcade);

    ClassicGames* classic = new ClassicGames();
    classic->add(retro);

    MainHall* hall = new MainHall();
    hall->add(classic);

    GameFest* fest = new GameFest();
    fest->add(hall);
}
    
```

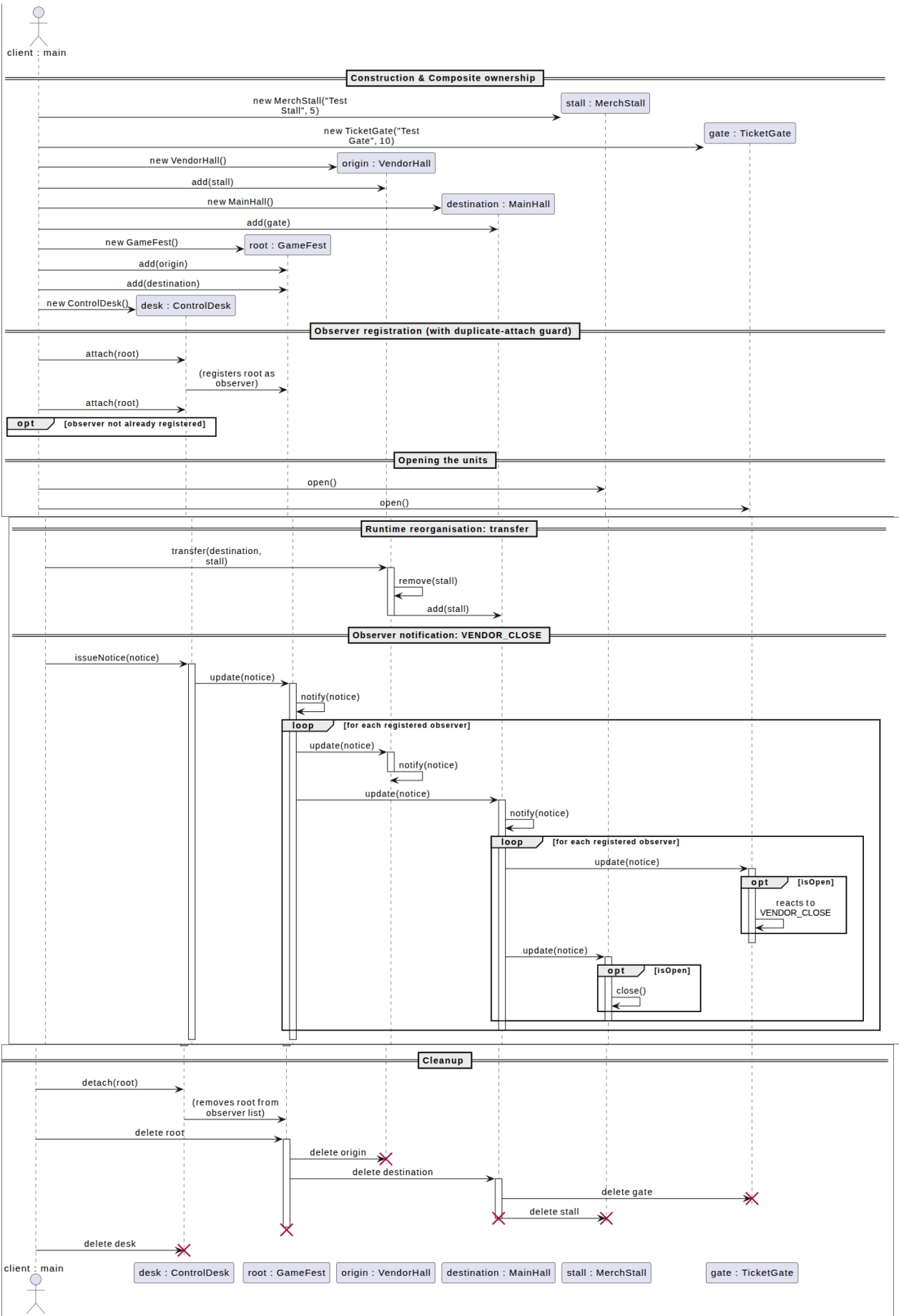
```
ControlDesk* desk = new ControlDesk();  
desk->attach(fest);
```

```
Notice open;  
open.type = OPEN;  
open.message = "Open the festival!";  
desk->issueNotice(open);
```

```
desk->detach(fest);  
delete fest;  
delete desk;
```

```
}
```

5.4) SD4 – Event Scenario



[diagram \(plantuml\)](#)

```
void sd4_transferDemo()
{
    MerchStall* stall = new MerchStall("Test Stall", 5);
    TicketGate* gate = new TicketGate("Test Gate", 10);

    VendorHall* origin = new VendorHall();
    origin->add(stall);

    MainHall* destination = new MainHall();
    destination->add(gate);

    GameFest* root = new GameFest();
    root->add(origin);
    root->add(destination);

    ControlDesk* desk = new ControlDesk();
    desk->attach(root);
    desk->attach(root);

    stall->open();
    gate->open();

    // reassignment mid-scenario
    origin->transfer(destination, stall);

    Notice close;
    close.type = VENDOR_CLOSE;
    close.message = "Vendors closing";
    desk->issueNotice(close);

    desk->detach(root);

    delete root;
    delete desk;
}
```

Task 6

Doxygen evidence (generated landing page or short screenshot/export showing successful generation)

The screenshot shows the 'Class List' page generated by Doxygen for a project named 'GameFest'. The page title is 'Class List' and it includes a search bar. A sidebar on the left shows the project structure with 'GameFest' expanded, showing 'Classes', 'Class List', 'Class Index', 'Class Hierarchy', 'Class Members', and 'Files'. The main content area lists 20 classes, structs, unions, and interfaces with brief descriptions:

- ArcadeRow**: An arcade row with all of the arcade machines, contained in the retro corner
- ControlDesk**: Concrete Subject acting as the event's central control point
- DemoStation**: A demo station for vendors, inside the vendor hall
- eSportsArena**: An eSports arena for tournament hosting, contained in the tournament wing
- EventComponent**: Common interface for every node in the EventFlow Composite tree
- EventGroup**: A composite grouping of EventComponents (e.g. the main hall)
- EventUnit**: Abstract base for leaf event units (stages, gates, stalls, etc.)
- GameFest**: Game Fest contains the main hall, the tournament wing and the vendor hall
- MainHall**: The main hall containing the main stage, the ticket gate and the retro corner of arcades
- MainStage**: The main stage inside the main hall
- MerchStall**: The merch stall inside the vendor hall
- Notice**: The payload passed in from a Subject to its Observers
- Observer**: Receives notifications from a Subject it is registered with
- Queue**: The queue inside the tournament wing
- RetroCorner**: Retro corner containing the arcade row, inside of the main hall
- Subject**: Maintains a list of registered observers and broadcasts notices Subject is the Subject role in the GoF Observer pattern. This Subject stores non-owning pointers: Observers must remain alive while registered and are responsible for detaching themselves before own destruction
- TicketGate**: The ticket gate inside the main hall
- TournamentWing**: Tournament wing contains the eSports arena and the queue
- VendorHall**: Vendor hall for different vendors' merch sales and demonstration stations

The bottom right corner indicates 'Generated by doxygen 1.16.1'.

The screenshot shows the 'Observer Class Reference' page generated by Doxygen for the 'GameFest' project. The page title is 'Observer Class Reference' and it includes a search bar. A sidebar on the left shows the project structure with 'GameFest' expanded, showing 'Classes', 'Class List', 'Class Index', 'Class Hierarchy', 'Class Members', and 'Files'. The main content area displays the 'Observer' class reference, including its description, preprocessor directives, inheritance diagram, and public member functions.

Observer Class Reference abstract

Receives notifications from a Subject it is registered with. More...

```
#include <Observer.h>
```

Inheritance diagram for Observer:

```
graph TD
    Observer[Observer]
    EventComponent[EventComponent]
    EventGroup[EventGroup]
    EventUnit[EventUnit]
    GameFest[GameFest]
    MainHall[MainHall]
    RetroCorner[RetroCorner]
    TournamentWing[TournamentWing]
    VendorHall[VendorHall]
    ArcadeRow[ArcadeRow]
    DemoStation[DemoStation]
    MainStage[MainStage]
    MerchStall[MerchStall]
    Queue[Queue]
    TicketGate[TicketGate]
    eSportsArena[eSportsArena]

    Observer --|> EventComponent
    EventComponent --|> EventGroup
    EventComponent --|> EventUnit
    EventGroup --|> GameFest
    EventGroup --|> MainHall
    EventGroup --|> RetroCorner
    EventGroup --|> TournamentWing
    EventGroup --|> VendorHall
    EventUnit --|> ArcadeRow
    EventUnit --|> DemoStation
    EventUnit --|> MainStage
    EventUnit --|> MerchStall
    EventUnit --|> Queue
    EventUnit --|> TicketGate
    EventUnit --|> eSportsArena
```

Public Member Functions

```
virtual ~Observer ()
    Virtual destructor.
virtual void update (Notice &notice)=0
```

The bottom right corner indicates 'Generated by doxygen 1.16.1'.

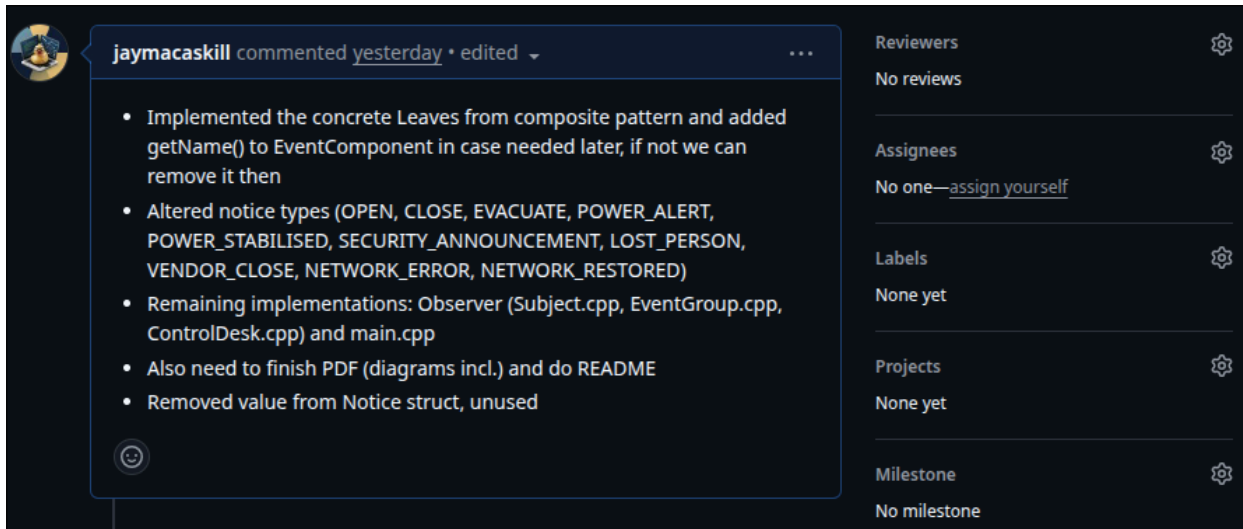
Task 7

GitHub repository link: <https://github.com/jaymacaskill/alt-f4> (currently private, will make public after submission deadline)

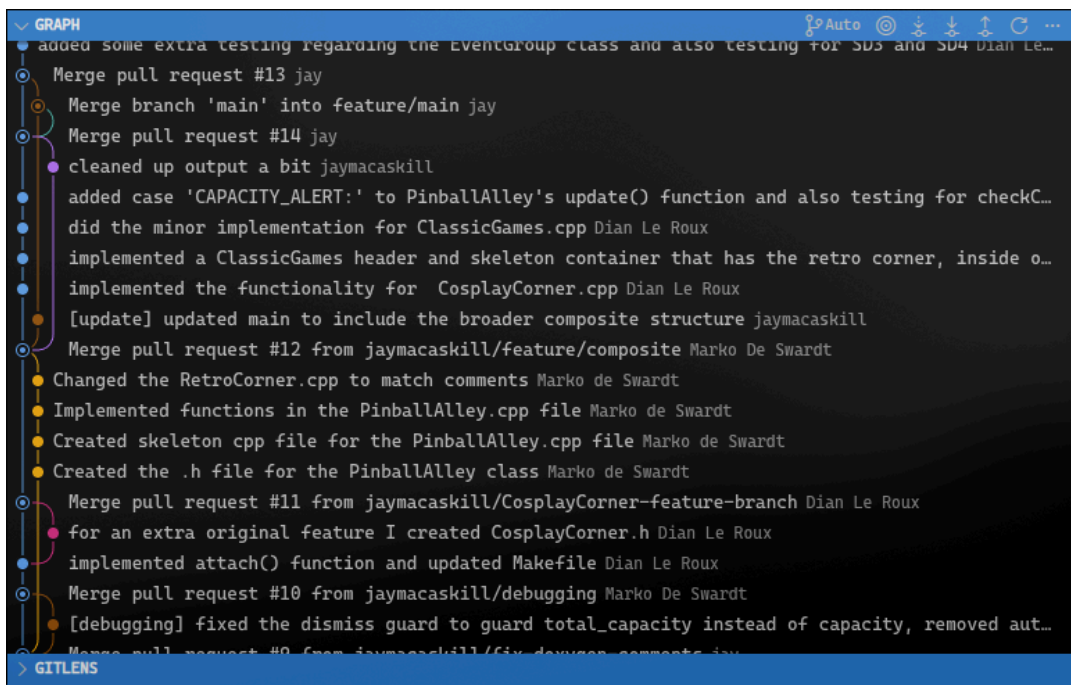
We made use of GitHub and branches, we later merged our branches to main once we were satisfied. We made use of meaningful commits with clear commit messages and made atomic use of commits as well. We had regular communication throughout the practical, balanced code and task division and reviews.

GitHub was effective for keeping our work organised, allowing us to work in parallel without ever stepping on each other's toes. We were able to keep the history clean and linear, which made it easy to track what was added and when.

Making use of pull requests enabled further confidence whilst coding, because pull requests can easily be rejected and reviewed by other team members. Here is an instance where we were able to make use of GitHub to clearly communicate what work was done and what was still outstanding, which was helpful to make collaboration more efficient, and changes clear.



Here we see how we were able to make use of branches to review each other's work and collaborate effectively.



Our team followed a rotating roles approach to ensure everyone contributed meaningfully. The table below shows the phases and roles of the team members during those phases.

Phase	Driver	Navigator	Tester
Planning	Dian	Marko	Jay
Composite Pattern	Marko	Jay	Dian
Observer Pattern	Dian	Marko	Jay
Integration	Jay	Dian	Marko

We lastly reviewed and constructed main.cpp together to ensure all features were demonstrated.

Team Contribution

Dian:

I implemented the Subject class in full (attach, detach, notify, the Observer pattern's registration and push-notification mechanism). Implemented most of EventGroup's Composite mechanics: the destructor, add(), remove(), and open(), plus the later getOccupancy()/checkCapacity() extension used for the Task 4.3 condition-based decision and its cascading CAPACITY_ALERT. Implemented ControlDesk::issueNotice(). Designed and implemented CosplayCorner (the Best Dressed competition, one of the three original features) and ClassicGames (an additional Composite nesting level). Added the EventGroup-focused unit tests and the SD3–SD4 demo functions in main.cpp. Led the correctness review of the submission PDF against the spec and the live code, catching several sections that had gone out of date as the design evolved. Co-developed the design rationale for the project's original features and Task 4.3's condition-based decision, including the reasoning for why CosplayCorner counts as real event specific functionality rather than a pattern driven difference, and the design of the capacity threshold alert mechanism.

Marko:

I have implemented a large share of EventGroup's Composite logic: close(), reportStatus(), getCapacity(), update() (the cascading re-broadcast that makes Observer and Composite work together), and transfer() (the Task 4.2 runtime reorganisation feature). Added CAPACITY_ALERT to the Notice enum and wired its case statements across Queue, TicketGate, ArcadeRow, DemoStation, eSportsArena, MainStage, and MerchStall. Created and implemented PinballAlley from scratch (header through full leaf behaviour). Fixed RetroCorner's implementation to match its documented design, and completed the Doxygen comment pass across the codebase.

Jay

- Completed the initial design including design rationale by collaborating with team to ensure ideas were uniformly agreed upon
- Created PDF and completed sections whilst work was occurring in the background to ensure uniform, balanced progress amongst the team
- Kept everyone on track regarding deadlines
- Completed the UML diagrams, excluding sequence diagrams
- Implemented EventUnit base class and concrete classes
- Implemented one of three original features
- Implemented EventGroup abstract classes
- Came up with scenario for main.cpp
- Responsible for documentation including:
 - README
 - Makefile
 - Doxyfile

- Setup the GitHub repository
 - Managed pull requests & reviews
- Implemented SD4 & SD2
- Aided the team with testing, debugging and review

Design Questions

Is EventControl itself part of the Composite or only a Subject/client collaborator?

ControlDesk is only a Subject and is not part of the Composite tree

Does every Composite observe its parent, or can some areas be structurally contained without observing notifications from that parent?

Every EventGroup observes its parent to enable cascading notifications, though this is managed separately from structural containment

Do all Leaves implement Observer, or only those interested in notifications?

All EventUnit leaves implement Observer because they all inherit from EventComponent, but they may ignore notices they don't care about

If a Composite owns a child, does it automatically register that child as an observer?

No, ownership and observer registration are managed separately; add() handles ownership and attach() handles observer registration, but when a child is added, it is convenient to also attach it as an observer, so that gets done, but it is not part of add(), attaching happens within add()

Can the same observer register with more than one subject?

Yes, an observer can register with multiple subjects

What happens if attach() is called twice with the same pointer?

The second attach() is ignored silently to prevent duplicate notifications

What happens when detach() cannot find the requested observer?

The detach() call is ignored silently

What exactly is stored as "current notice" or subject state?

The Notice struct stores the notice type and a message string

Who initiates cascading notification after an observing Composite receives update()?

The EventGroup itself calls notify() inside its update() method to re-broadcast to its children

What is the order of notifications, and does that order matter in your event?

Notifications follow registration order, but the order does not matter for our event

What happens if an observer changes registration while notification is in progress?

notify() copies the observer list before iterating, so changes during notification are safe

Can an operational unit move between Composite parents?

Yes, via the `transfer()` method

Who owns it during the transfer?

The new parent owns it after the transfer, and the old parent releases ownership

How do you prevent double deletion if a unit is transferred?

`remove()` returns the unit without deleting it, and ownership is explicitly transferred to the new parent via `add()`

What does aggregate capacity/strength/status mean for your particular event?

For an `EventGroup`, aggregate capacity is the sum of all children's capacities